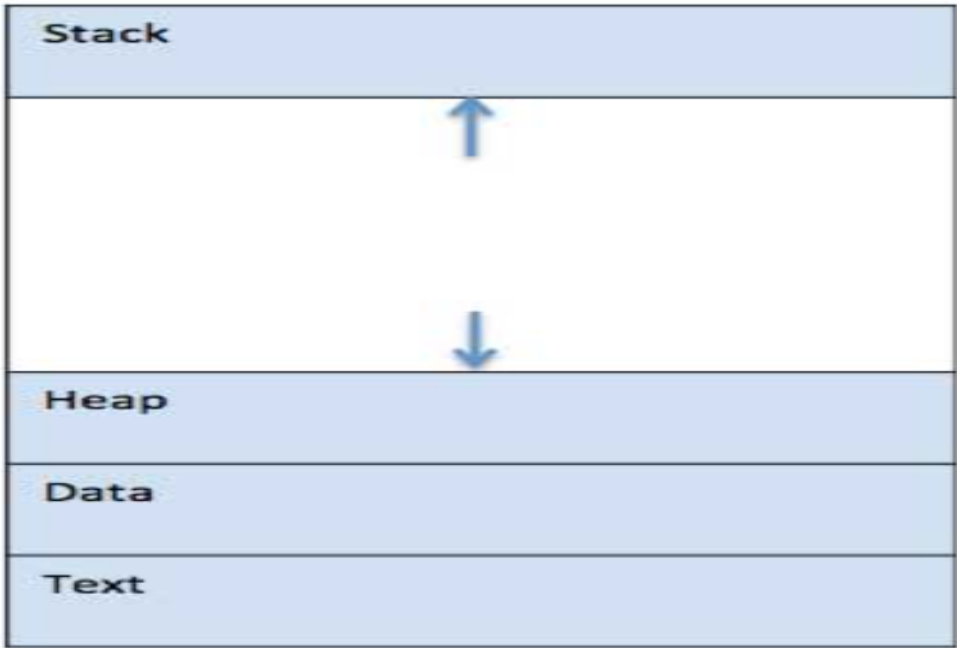
Process

A process is basically a program in execution. The execution of a process must progress in a sequential fashion.

- *A process is defined as an entity which represents the basic unit of work to be implemented in the system.*

- To put it in simple terms, we write our computer programs in a text file and when we execute this program, it becomes a process which performs all the tasks mentioned in the program.
- When a program is loaded into the memory and it becomes a process, it can be divided into four sections: stack, heap, text and data. The following image shows a simplified layout of a process inside main memory



Attributes of a Process

A process has several important attributes that help the operating system manage and control it. These attributes are stored in a structure called the

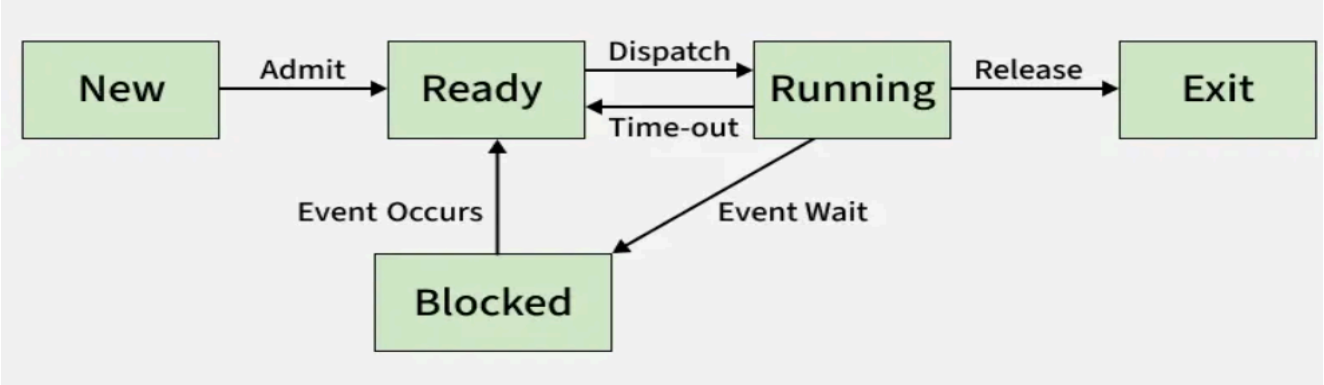
Process Control Block (PCB) (sometimes called a task control block). The PCB keeps all the key information about the process, including:

1. **Process ID (PID)**: A unique number assigned to each process so the operating system can identify it.
2. **Process State**: This shows the current status of the process, like whether it is running, waiting, or ready to execute.
3. **Priority and other CPU Scheduling Information**: Data that helps the operating system decide which process should run next, like priority levels and pointers to scheduling queues.
4. **I/O Information**: Information about input/output devices the process is using.
5. **File Descriptors:** Information about open files and network connections.
6. **Accounting Information**: Tracks how long the process has run, the amount of CPU time used, and other resource usage data.
7. **Memory Management Information**: Details about the memory space allocated to the process, including where it is loaded in memory and the structure of its memory layout (stack, heap, etc.).

States of a Process in Operating Systems

A process in an operating system passes through multiple states as it begins execution, waits for resources, gets scheduled, runs, and eventually finishes. These stages collectively describe the complete lifecycle of a process.

The Five-State Model



Five-State Model

The five states are:

- **New:** The process has just been created. It hasn't started running yet, but its PCB (Process Control Block) is prepared.
- **Ready:** The process is loaded into memory and prepared to run as soon as the CPU becomes free.
- **Running:** The CPU is currently executing this process. With a single CPU system, only one process can be in this state at a time.
- **Blocked/Waiting:** The process cannot continue execution because it is waiting for an event like I/O completion or data input.
- **Exit/Terminate:** The process has completed its execution or has been stopped, and the operating system removes it from memory.

Important State Transitions

- **New → Ready:**
  A process is created, resources are allocated, and it is loaded into main memory.
- **Ready → Running:**
  The scheduler selects a ready process and assigns the CPU to it.
- **Running → Blocked (Waiting):**
  The process must wait for an event or resource (e.g., I/O, user input, system call).
- **Blocked → Ready:**
  The event completes or the needed resource becomes available, so the process is ready to run again.
- **Running → Ready:**
  The OS preempts the running process—often because a higher-priority process becomes ready.
- **Running → Terminated:**
  The process completes its execution or is forcefully stopped.

Process Image

- process image is a complete collection of information needed to represent a process in memory and manage its execution.
- It commonly includes the program code, data, stack, heap and operating-system control information.

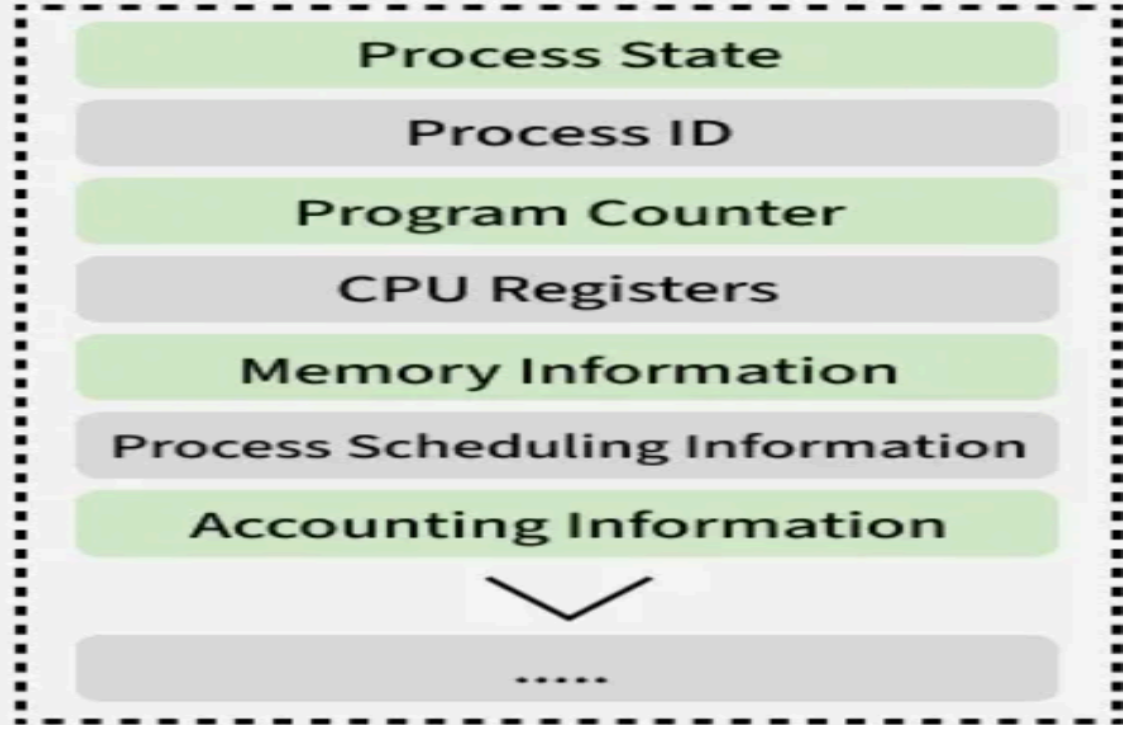
Code / text: machine instructions of the program.

• Data: global and static data used by the program.

• Heap: dynamically allocated memory.
• Stack: function calls, local variables and execution-related information.
• PCB: operating-system information describing and controlling the process.

A Process Control Block (PCB)

- A Process Control Block (PCB) is a data structure used by the operating system to keep track of process information and manage execution. It helps the OS monitor and control process execution.
- PCB as the **identity card or record file of a process**.
- When a process is created, the OS creates a PCB for it. When the process finishes, its PCB is removed.
- Each process is given a unique Process ID (PID) for identification.
- During a state transition, the OS updates the PCB with the latest execution data.



The following are the data items –

- **Process Number or ID** – This shows the identification number of the particular process.
- **Process State** – This specifies the process state i.e. new, ready, running, waiting or terminated.
- **Program Counter** – This contains the address of the next instruction that needs to be executed in the process.
- **Registers** – This specifies the registers that are used by the process. They include accumulators, index registers, stack pointers, general purpose registers etc.
- **List of Open Files** – These are the different files that are associated with the process.
- **CPU Scheduling Information** – The process priority, pointers to scheduling queues etc. is the CPU scheduling information that is contained in the PCB. This may also include any other scheduling parameters.

- **Memory Management Information** – The memory management information includes the page tables or the segment tables depending on the memory system used. It also contains the value of the base registers, limit registers etc.
- **I/O Status Information** – This information includes the list of I/O devices used by the process, the list of files etc.
- **Accounting Information** – The time limits, account numbers, amount of CPU used, process numbers etc. are all a part of the PCB accounting information.
- **Location of The Process Control Block** – The process control block is kept in a memory area that is protected from the normal user access. This is done because it contains important process information. Some of the operating systems place the PCB at the beginning of the kernel stack for the process as it is a safe location.

Process Management Tasks That Utilize PCB

Process Control Blocks play an important role to perform various process management activities since they store the vital information related to the processes.

a)**Process Scheduling**→ The operating system uses the contents of PCB such as process priority, process state, and resources utilized to schedule the processes from the ready queue. By assessing the priority and other related information, the scheduler determines the sequence of process execution.

b)Multitasking→ The operating system can multitask and run various processes simultaneously with the help of information stored in PCBs like resource allocation, process scheduling, and process synchronization.

c)**Resources Sharing**→The information stored in the PCB includes the resources allocated to a process, like open files, shared variables, common tables and memory space. When a new process requests resources, the OS searches the allocated processes in the PCBs, checks whether they are shareable or not and allocates the resources accordingly. Thus PCBs play an important role in resource sharing.

d)Context switching→Context switching is the process where the CPU stops running one process, saves its current state, and loads the saved state of another process so that multiple processes can share the CPU effectively.

Execution of the Operating System

User Mode and Kernel Mode in Operating System

A computer has **important resources** such as:

- CPU
- RAM

- Hard disk/SSD
- Keyboard
- Printer
- Network devices

If every application could directly control these resources, it could cause **errors, security problems, or damage**.

So the Operating System provides **two modes of execution**:

1. **User Mode**
2. **Kernel Mode**

User Mode

User Mode is the mode in which **normal application programs run**.

Examples:

- Google Chrome
- MS Word
- Calculator
- VLC
- Your Python program

These applications have **limited permissions**.

User Mode is a restricted mode in which application programs execute without direct access to critical hardware and system resources.

For example, suppose Chrome wants to read a file from your hard disk.

Chrome cannot directly control the hard disk.

```
Chrome
   ↓
Request to Operating System
   ↓
Kernel
   ↓
Hard Disk
```

Kernel Mode

Kernel is the core (main part) of an Operating System that manages the computer's hardware and provides services to application programs.

Think of the **kernel as the manager of the computer**.

Kernel Mode is the mode in which the **OS kernel executes with high privileges**.

The kernel can access and control important system resources such as:

- CPU
- Memory
- Disk
- I/O devices
- Hardware

Simple definition:

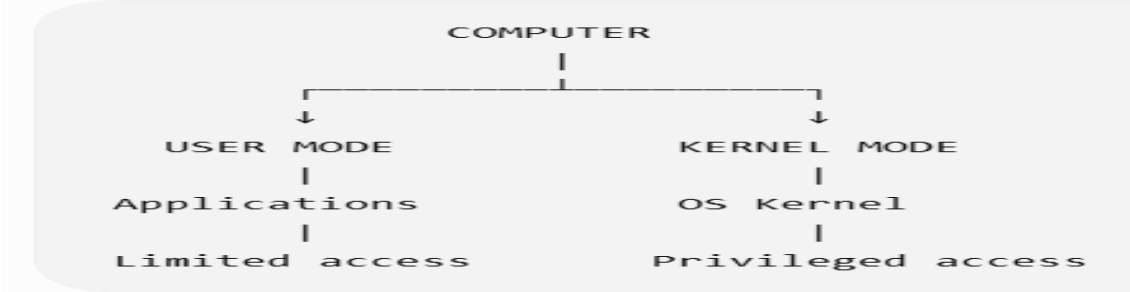
> **Kernel Mode is a privileged mode in which the Operating System kernel can directly access and manage hardware and protected system resources.**

Why do we need two modes

Imagine if a normal application could directly access RAM or the hard disk.

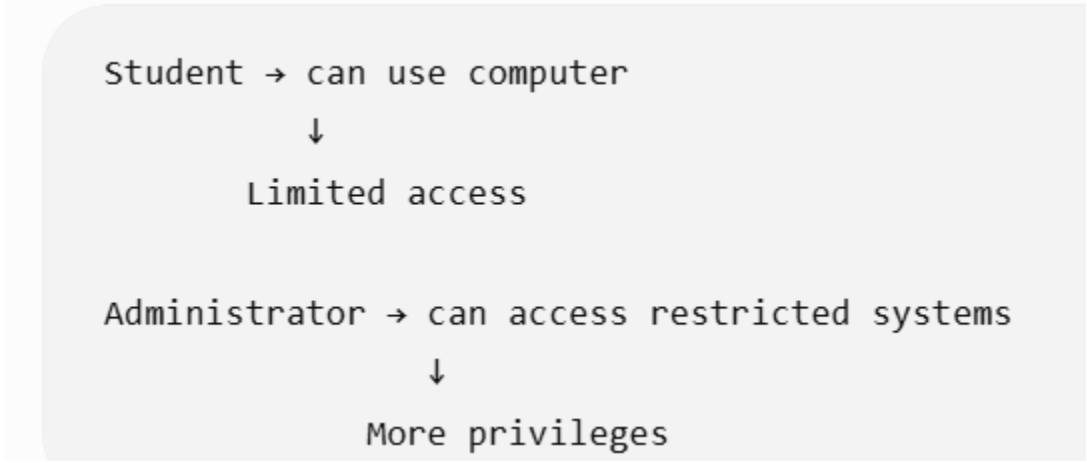
For example, a badly written program could accidentally modify another program's memory.

That could cause the system to crash.



Think about a **college laboratory**.

Students can use the computers, but they cannot enter the server room and change important equipment.



User Mode = Limited permission, Kernel Mode = High permission

User Mode vs Kernel Mode

User Mode	Kernel Mode
Used by application programs	Used by OS kernel
Restricted privileges	High privileges
Cannot directly access protected hardware	Can access protected resources
Safer for the system	More powerful
Example: Chrome, Word	Example: OS kernel
Uses system calls to request OS services	Provides OS services

Thread in Operating System

Process

A **process** is a **program that is currently running**.

For example:

- You open **Google Chrome** → Chrome becomes a process.
- You open **MS Word** → Word becomes a process.
- You run a Python program → the running Python program is a process.

Thread

- A thread is a small sequence of instructions that performs a particular task inside a process.
- One process can contain **one or more threads**.
- Threads are used to perform **multiple tasks within the same process**.
- **A thread is a single sequence stream within a process and is called a lightweight process because it is smaller and faster. It allows multiple tasks to run simultaneously, improving program efficiency.**
- **Threads are used to achieve parallelism. It divides a process into multiple threads which have independent paths of execution while sharing the same common memory, space, and resource. For example: tabs of a web browser, text editors and many more.**
- Each thread has its own program counter, register set, and stack space.
- Threads share code, data, and operating system resources within the same process.

Process = a complete working application
Thread = a task being performed inside that application

Simple real-life example

Think about a **college**:

- **College → Process**
- **Different departments/tasks → Threads**

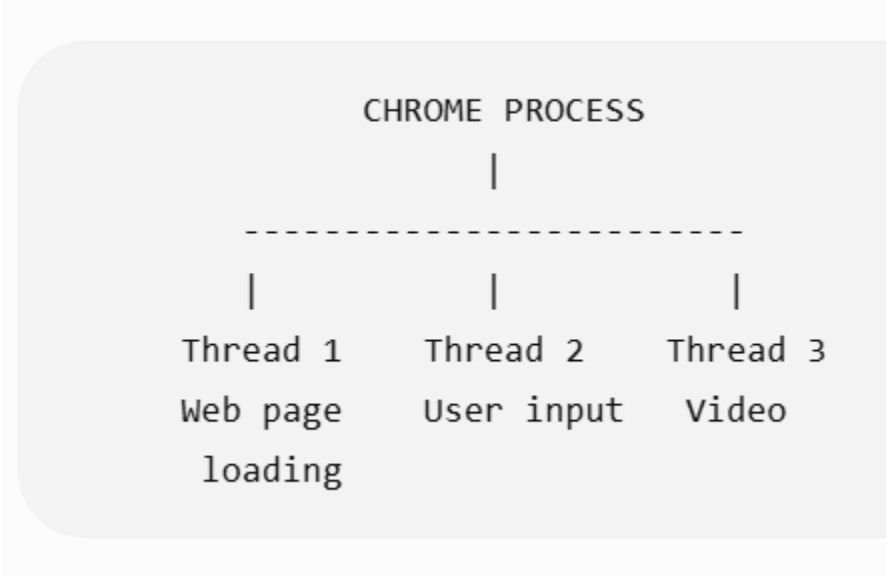
Example: Web Browser

Suppose you open Google Chrome.Chrome is a **process**.Inside Chrome, different tasks may happen:

- One thread loads a webpage.
- Another thread handles user input.
- Another thread plays a video.
- Another thread downloads a file.

Chrome → Process

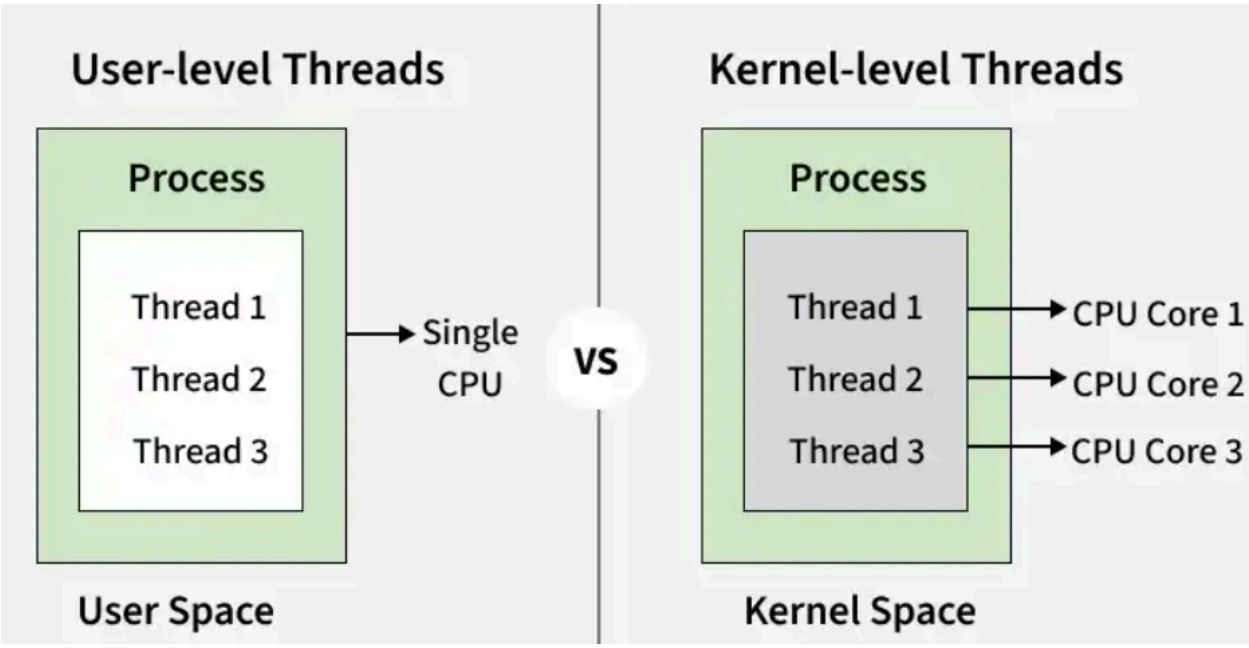
Inside Chrome:



Types of Threads

There are two types of threads

- User Level Threads
- Kernel Level Threads



User Level Threads(ULTs)

- Managed entirely in user space using a thread library; the kernel is unaware of them.
- Switching between ULTs is fast since only the program counter, registers, and stack need to be saved/restored.
- Do not require system calls for creation or management, making them lightweight.
- If one thread makes a blocking system call, the entire process (all threads) is blocked.
- Scheduling is done by the application itself, which may not be as efficient as kernel-level scheduling.
- Cannot fully utilize multiprocessor systems because the kernel schedules processes, not individual user-level threads.
- Examples of user-level thread: POSIX Pthreads, Java green threads, GNU Portable Threads (GNU Pth) etc.

Kernel-Level Threads (KLTs)

- Managed directly by the operating system kernel; each thread has an entry in the kernel's thread table.
- The kernel schedules each thread independently, allowing true parallel execution on multiple CPUs/cores.
- Handles blocking system calls efficiently; if one thread blocks, the kernel can run another thread from the same process.
- Provides better load balancing across processors since the kernel controls all threads.
- Context switching is slower compared to ULTs because it requires switching between user mode and kernel mode.
- Implementation is more complex and requires frequent interaction with the kernel.
- Large numbers of threads may add extra load on the kernel scheduler, potentially affecting performance.

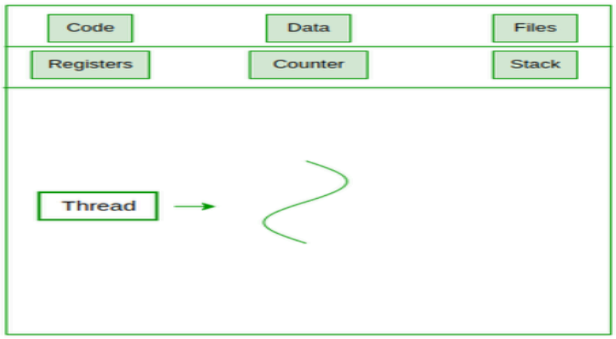
Advantages of Thread

Threads are needed in modern operating systems and applications because they:

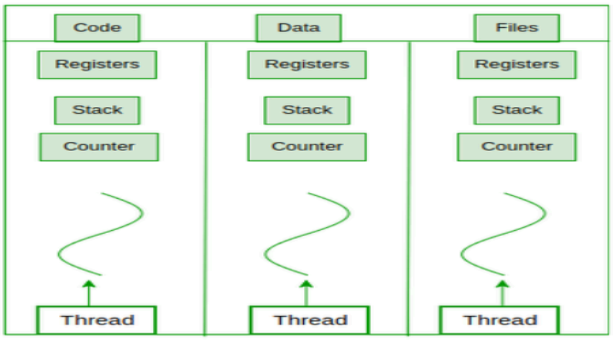
- **Improve Performance:** Threads allow multiple tasks to run at the same time (parallel or interleaved), making programs execute faster.
- **Increase Responsiveness:** If one thread is busy, another can handle user actions, keeping the application responsive.
- **Enable Concurrency:** Multiple operations like saving files, processing data, and handling user input can happen simultaneously.
- **Better CPU Utilization:** On multi-core systems, threads can run on different cores, improving overall system performance.
- **Efficient Resource Sharing:** Threads share the same memory and resources within a process, making communication faster and reducing overhead.

Multithreading

- In single-core systems, it creates an illusion of parallelism; in multi-core systems, threads can execute truly in parallel across different cores.
- Each thread has its own program counter, register set, and stack space.
- Threads share code, data, and operating system resources within the same process.



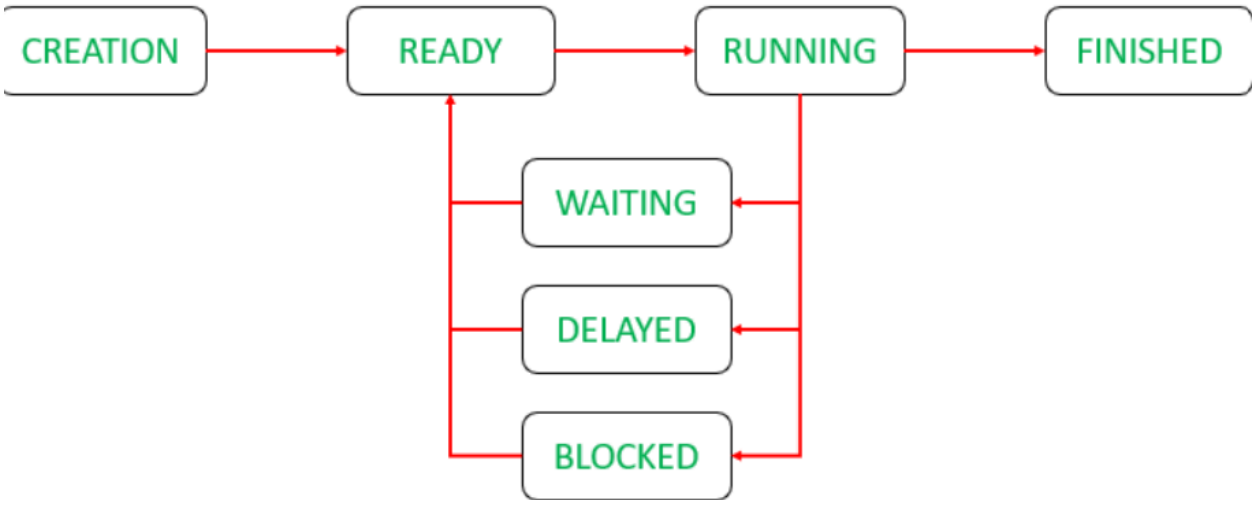
Single Threaded Process



Multi Threaded Process

Thread States in Operating Systems

1. Creation
2. Ready
3. Running
4. Waiting
5. Delayed
6. Blocked
7. Finished



1. When an application is to be processed, then it creates a thread.
2. It is then allocated the required resources(such as a network) and it comes in the **READY** queue.
3. When the thread scheduler (like a process scheduler) assign the thread with processor, it comes in **RUNNING** queue.
4. When the process needs some other event to be triggered, which is outsides it's control (like another process to be completed), it transitions from **RUNNING** to **WAITING** queue.
5. When the application has the capability to delay the processing of the thread, it when needed can delay the thread and put it to sleep for a specific amount of time. The thread then transitions from **RUNNING** to **DELAYED** queue. An example of delaying of thread is snoozing of an alarm. After it rings for the first time and is not switched off by the user, it rings again after a specific amount of time. During that time, the thread is put to sleep.
6. When thread generates an I/O request and cannot move further till it's done, it transitions from **RUNNING** to **BLOCKED** queue.
7. After the process is completed, the thread transitions from **RUNNING** to **FINISHED**.